

Tema 3 — Monitores

Fuentes: Teórica 03-Monitores, apunte de la Práctica 3 ("De semáforos a monitores"), Guía 3. **Ejercicio 3 del parcial.**

1. Por qué monitores

Los semáforos son el "gato" de la sincronización: operaciones sueltas que podés olvidarte de poner, poner al revés o desparramar por todo el código. Un monitor no resuelve nada que un semáforo no pueda resolver, pero **te ordena**. Separa dos cosas:

1. **Exclusión mutua automática:** todos los métodos del monitor se ejecutan de a uno. No escribís ningún lock.
2. **Espera condicional explícita:** "esperá hasta que *esta condición* sea verdadera", escrita como un booleano sobre variables reales.

En criollo: una oficina con una sola persona atendiendo (el lock implícito). Si te falta algo para seguir, te sentás en la sala de espera que corresponda (una variable de condición) y dejás pasar al siguiente. Cuando alguien cambia algo que te interesa, te avisa (signal).

2. Componentes

- **Datos privados + métodos** (se ejecutan en exclusión mutua) + **variables de condición**.
- **Una variable de condición es una COLA de threads dormidos** (no guarda ningún valor). Tiene:
 - `wait(c)` : **siempre** te duerme, y al mismo tiempo **suelta el lock** (si no lo soltara, nadie podría entrar a cambiar lo que esperás).
 - `signal(c)` : despierta a uno. **Si no hay nadie esperando, no hace nada:** no queda "guardado" como el permiso de un semáforo.
 - `signalAll(c)` : despierta a todos.
 - `empty(c)` : ¿hay alguien esperando?

Semáforo	Variable de condición
<code>acquire</code> puede no bloquear (depende del contador)	<code>wait</code> siempre bloquea
<code>release</code> siempre tiene efecto (suma o despierta)	<code>signal</code> sin nadie esperando se pierde
el contador y la espera están mezclados en un número	la condición la escribís vos con un <code>if / while</code>

3. Hoare vs Mesa (va a estar en el parcial seguro) ★

Después de un `signal` hay dos threads que "quieren estar adentro": el que señalizó (**S**) y el despertado (**W**). Además están los que esperan para entrar por primera vez (**E**). La disciplina define quién sigue.

	Hoare (signal y espera urgente, IRR)	Mesa (signal y continúa) — Java
Prioridad	$E < S < W$	$E = W < S$
Qué pasa	el despertado corre ya mismo , con el lock	el que señalizó sigue; el despertado va a competir por el lock con los que recién llegan
¿La condición sigue valiendo cuando el despertado corre?	Sí , garantizado	No necesariamente : otro puede haberla cambiado
Cómo se espera	<code>if (!cond) wait(c)</code> alcanza	<code>while (!cond) wait(c)</code> siempre

Además, en la vida real hay **spurious wakeups**: el thread se despierta sin que nadie haya hecho signal. Es otra razón independiente para usar `while` (en el parcial suelen decir "no hay spurious wakeups", pero con Mesa el `while` hace falta igual).

Consecuencias de Mesa:

- **No es fair**: uno que recién llega le puede "robar" la condición al despertado.
 - Arreglo: **pasar el testigo**. El que señaliza deja el estado ya asignado al despertado. Ejemplo, el semáforo fair: `signal() { if (empty(c)) sv++; else signal(c); }` y `wait() { if (sv == 0) wait(c); else sv--; }`. El permiso nunca queda "suelto" para que alguien se lo robe.
- **Lost wake-up**: si señalizás solo en una transición puntual (por ejemplo `if (cant == 1) signal()`), podés dejar dormido a alguien que ya podía seguir.
 - Regla: **señalizá cada vez que cambia el estado**.
- **Colas no FIFO + Mesa** → **no hay orden de llegada**. Si el enunciado pide "que todos lleguen tarde o temprano", hay que imponer el orden a mano con **tickets**.

4. Ejemplos de la teórica, a nivel patrón

- **Semáforo con monitor**: `while (sv == 0) wait(noCero); sv-- / sv++; signal(noCero)`.
- **Buffer acotado**: `while (cant == N) wait(hayEspacio) / while (cant == 0) wait(conDato)`. Cada operación señaliza a la otra condición.
- **Filósofos**: `forks[i]` = cuántos tenedores libres tiene el filósofo *i*. `takeForks` espera `forks[i] == 2` y toma **los dos juntos**, atómicamente, así que no hay deadlock. Al soltar, despierta a los vecinos que quedaron con 2.
- **Lectores-Escritores**:
 - Una sola condición: `startRead: while (writer) wait; startWrite: while (writer || readers > 0) wait`. Al salir, **signalAll**: con `signal` podrías despertar a un lector cuando el que podía seguir era un escritor.
 - Dos condiciones `readOk / writeOk`: el lector hace `signal(readOk)` en cascada para ir despertando al resto de los lectores. Al terminar, el escritor elige a quién despertar según la prioridad que quieras.

5. El apunte de la práctica 3 (leelo entero, es muy bueno)

1. **MonitorCasero**: un monitor armado con semáforos.
 - `mutex = Sem(1)` + una cola de semáforos privados.

- `esperar()` : me pongo en la cola, suelto el mutex, me duermo en **mi** semáforo, y al despertar vuelvo a pedir el mutex.
- `avisar()` : libera al primero de la cola. Pero el mutex lo sigue teniendo el que avisó → eso es exactamente **Mesa**.

2. Barrera con monitor:

```
miFase = fase; count++
if (count == n) { count = 0; fase++; notifyAll() }
else while (fase == miFase) wait()
```

La **fase** (o "generación") reemplaza al segundo molinete de la versión con semáforos.

3. Elecciones del CoDep (patrón cliente/servidor + ticket):

- Tres condiciones: `hayLlegada` y `terminoVoto` las espera la secretaria; `puedeEntrar` la esperan los estudiantes.
- Para respetar el orden, cada estudiante saca un **ticket** y espera `while (llamando != miTicket)` `wait(puedeEntrar)`. La secretaria hace `llamando++ + signalAll`.
- Con `synchronized` hay **una sola cola**, así que `notify()` puede despertar al que no era ("despertar perdido por multiplexado") → usar `notifyAll`, o `ReentrantLock` con varias `Condition`.

4. Mostrador (los fumadores, versión monitor):

- Invariante: "hay 0 o 2 elementos presentes".
- `reponer` espera que el mostrador esté vacío; `juntarElementos(a, b)` espera `presente[a] && presente[b]`.
- La condición compuesta que con semáforos daba deadlock, con un monitor es un simple `while`.

5. Thread pool:

- Es un productor-consumidor de tareas. **La tarea se ejecuta afuera del lock**; si no, los workers se ejecutarían de a uno.
- Acá `notify` alcanza porque todos los workers son intercambiables.
- **Promesa**: se resuelve una sola vez; `signalAll` porque varios pueden estar esperando el mismo resultado.

6. Java

- **Opción 1**: métodos `synchronized + wait()`, `notify()`, `notifyAll()`. Hay **una sola cola implícita** por objeto, y es Mesa. `wait` afuera de `synchronized` tira `IllegalMonitorStateException`.
- **Opción 2**: `ReentrantLock + lock.newCondition()` (tantas condiciones como quieras): `await()`, `signal()`, `signalAll()`, con `unlock()` **en un finally**.

7. Plantilla mental para cualquier ejercicio de monitores

1. ¿Qué **estado** necesito? (contadores, booleanos, estado del recurso, ticket o turno)
2. Para cada operación: ¿**qué condición** tiene que valer para seguir? → `while (!condición) wait(c)`.
3. Actualizo el estado.
4. ¿**A quién** le puede servir este cambio? → `signal` si es uno cualquiera de un grupo intercambiable; `signalAll` si no lo son o si se liberan varios.
5. ¿Piden orden o que nadie espere para siempre? → **ticket FIFO**.
6. Justificar:

- **mutex:** es automático en el monitor;
- **no deadlock:** todo el que espera tiene a alguien que lo va a despertar;
- **no inanición:** con tickets o por qué no hace falta;
- **y con Hoare...:** cómo cambiaría.

8. Guía 3 — qué patrón entrena cada ejercicio

Ej	Patrón
1	Hoare vs Mesa con una traza (tipo inciso b)
2	secuenciador ternario: variable de turno cíclica
3	barrera (uso único y reutilizable → fase)
4	atrapador: liberar N juntos; "¿se cuela uno que no esperó?" → generación
5	peluquería: cliente/servidor con varias condiciones (y con una sola)
6	charlas: estado de la sala (abierta / en curso), capacidad
7	apuestas: "no podés apostar dos veces seguidas" (quién apostó último)
8	pizzas: condición compuesta (una grande o dos chicas)
9	bote: costa + capacidad + autorización; (b) orden de llegada → ticket
10	recursos: tomar k; (c) "no perjudicar a los que piden muchos" = inanición → ticket FIFO. Es el mismo problema que las mesas del simulacro ★